

Validação Paralela de Clusters em GPU

ETAPA 2 – MODELAGEM

Métricas: Índice de Dunn, Coeficiente de Silhueta e Davies-Bouldin

Equipe

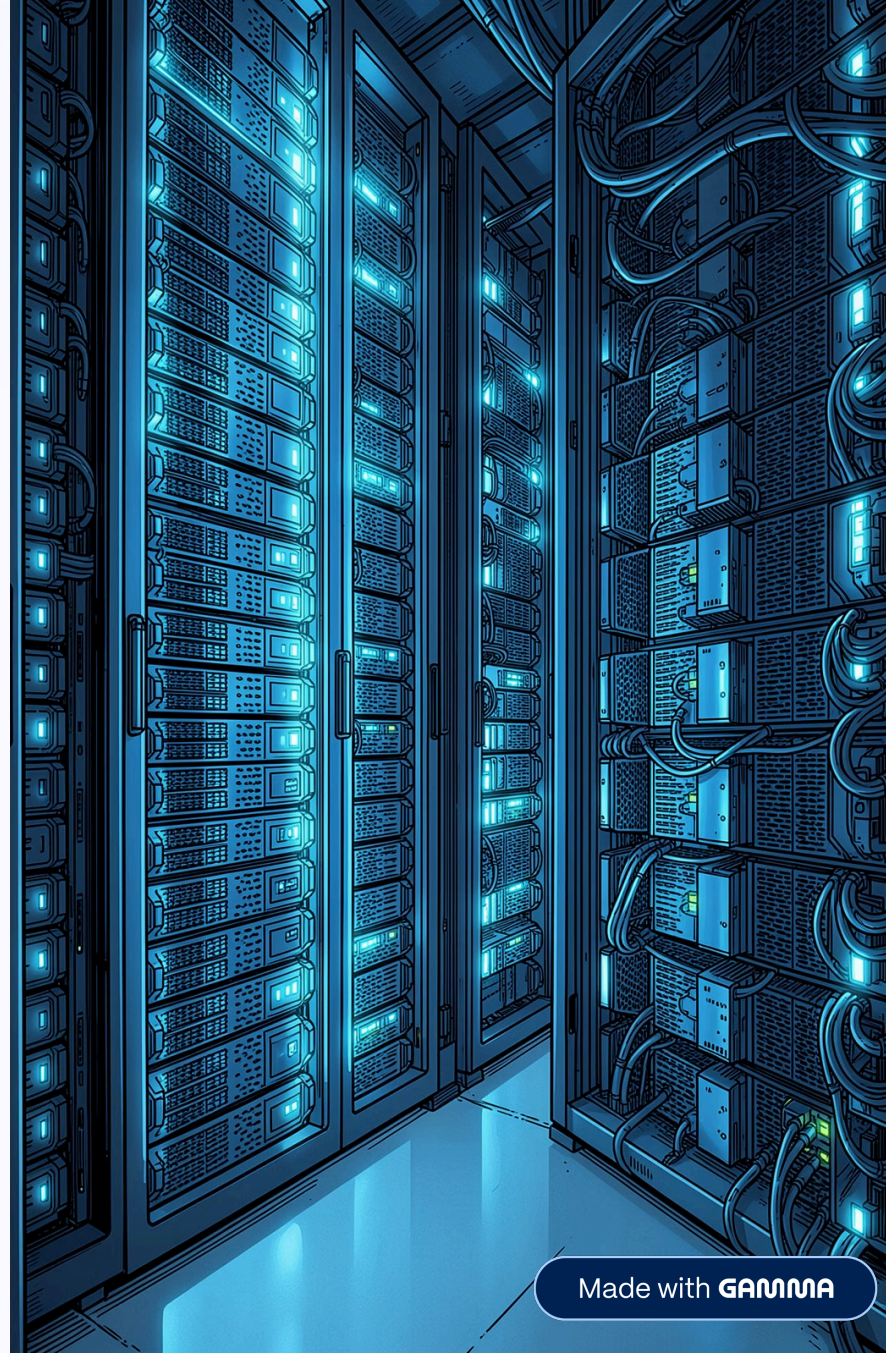
- Henrique M. M. Miranda – 202405479
- Cindy Stephanie Gomes Rabelo – 202403898
- Eduardo Dias Peixoto – 202010395
- Luiany Goncalves Carvalho – 202303351

Informações Acadêmicas

Disciplina: Computação de Alto Desempenho

Professor: Ricardo Augusto Pereira Franco

Instituição: UFG – Instituto de Informática



O Gargalo de Validação de Clusters

O Problema

Algoritmos de clustering (K-Means, DBSCAN) agrupam os dados, mas validar a qualidade destes agrupamentos exige calcular a proximidade e isolamento dos pontos.

O Gargalo

Métricas como Dunn e Silhueta exigem o cálculo das distâncias euclidianas par-a-par de todos os pontos – complexidade **$O(N^2)$** .

Impacto em Big Data

Para datasets com dezenas de milhares ou milhões de pontos, o cálculo sequencial na CPU torna-se inviável devido ao crescimento quadrático.

Índice de Dunn

Maximizar separação em relação à dispersão intra-cluster.

Coeficiente de Silhueta

Ajuste local de cada ponto em relação ao cluster vizinho mais próximo.

Davies-Bouldin

Razão média entre dispersões internas e distâncias entre centroides.

Metodologia de Paralelização em GPU (CUDA)



Matriz Euclidiana

Redução Dunn

Redução CPU

A. Matriz de Distâncias Euclidiana

Thread (x, y) calcula a distância $d(x_i, x_j)$. Lançado em grid 2D com blocos de **16 × 16 threads**, cobrindo toda a matriz de distâncias de forma massivamente paralela.

B. Índice de Dunn – Redução Paralela

Lançamos **N blocos de 256 threads** (um bloco por ponto/linha da matriz). As threads varrem a linha em paralelo e realizam **Redução em Árvore com Memória Compartilhada** (`__shared__`) para encontrar a maior distância intra-cluster e a menor inter-cluster. A CPU realiza apenas a redução global linear $O(N)$.

Metodologia Paralela: Silhueta e Davies-Bouldin

C. Coeficiente de Silhueta

Shared Memory Dinâmica

- Um bloco de 256 threads por ponto i
- Memória compartilhada dinâmica (`extern __shared__ double shared_sums[]`) armazena a soma das distâncias de i para cada um dos K clusters
- Cada thread varre os pontos e acumula a distância para o cluster de destino
- Threads realizam redução local e a thread 0 calcula $s_i = (b_i - a_i) / \max(a_i, b_i)$

D. Davies-Bouldin

Operações Atômicas no Device

- Centroides e dispersões são acumulados em paralelo via `atomicAdd` no device
- O cálculo da razão R_{ij} é resolvido em paralelo por cluster
- Minimiza conflitos de escrita entre threads concorrentes
- Resultado final consolidado com eficiência máxima de hardware

- ❗ Ambas as abordagens exploram a hierarquia de memória da GPU – registradores, memória compartilhada e memória global – para maximizar o throughput computacional.

Validação Numérica Rigorosa

Para garantir a absoluta corretude do cálculo paralelo, validamos as implementações em **três camadas de testes independentes**:

1

Caso Analítico Estático – Índice de Dunn

Como o scikit-learn não implementa Dunn, criamos um caso de teste analítico com 4 pontos conhecidos (Dunn esperado de **2.0**). CPU e GPU obtiveram **2.0** – Erro = **0.00e+00**.

2

Comparação com Scikit-learn (Ground Truth)

Validamos Silhueta e Davies-Bouldin contra o scikit-learn oficial para Iris ($N=150$). Resultados bateram com precisão de máquina – diferença de $\sim 2.28 \times 10^{-9}$.

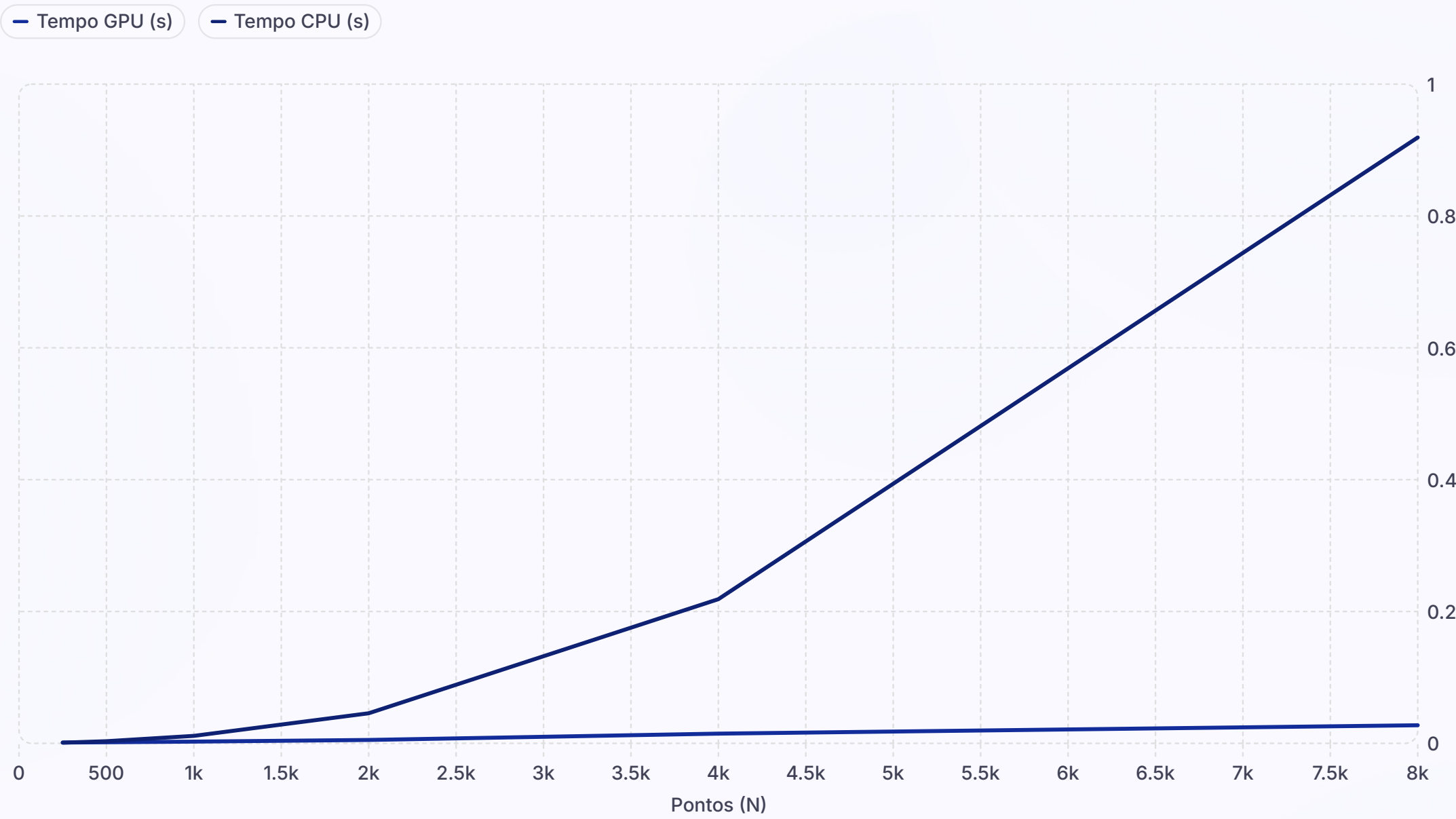
3

Equivalência CPU $\equiv$ GPU

Validamos todas as métricas em todos os tamanhos de datasets do benchmark. Ambas as execuções coincidiram em **100% dos testes**.

Experimentos e Resultados (CPU vs GPU)

Códigos testados sobre dados sintéticos (blobs) de dimensão **D=4** e **K=5** clusters. Comparação entre CPU sequencial e GPU paralela (NVIDIA T4 no Colab):



33.51x

Speed-up Máximo

Atingido com 8.000 pontos na GPU NVIDIA T4.

100%

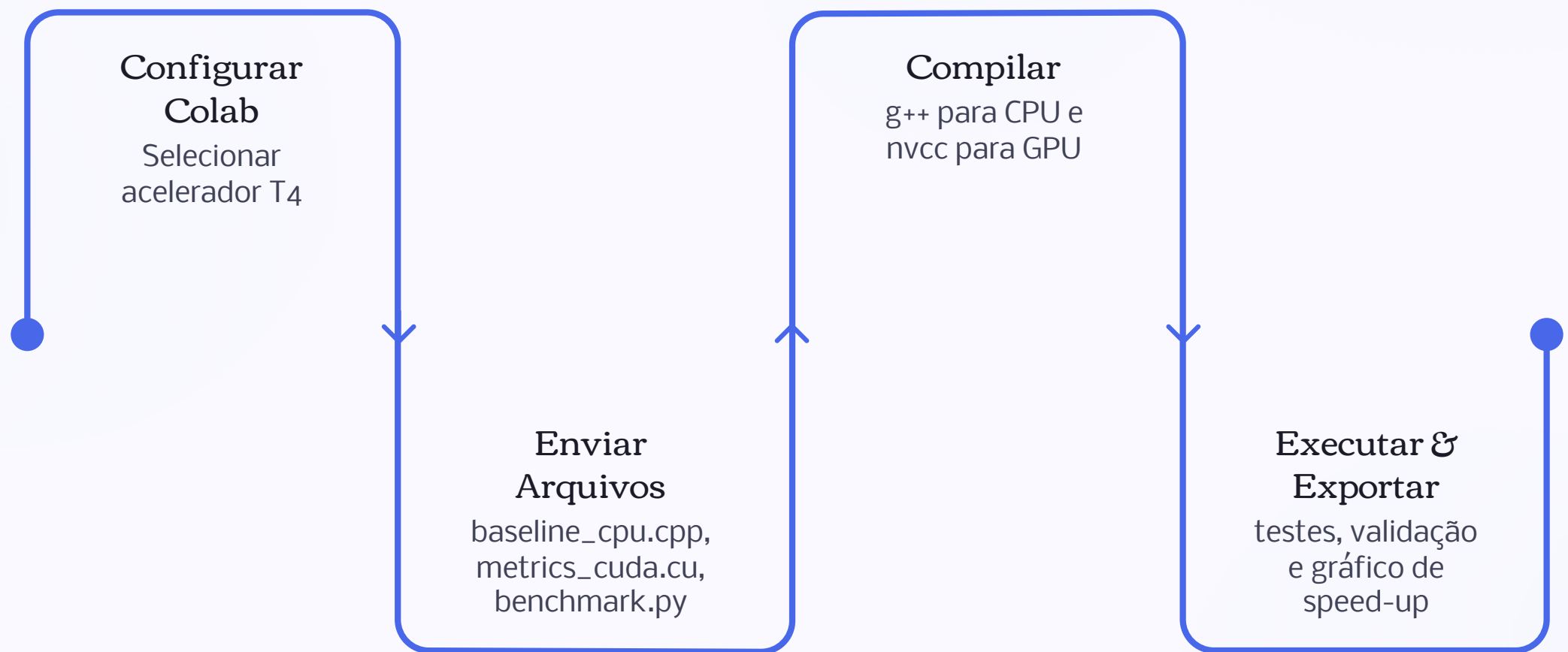
Corretude

CPU e GPU coincidiram em todos os testes de benchmark.

Pontos (N)	Tempo CPU (s)	Tempo GPU (s)	Speed-up	Corretude
250	0.0010	0.0012	0.87X	100% Match
500	0.0032	0.0016	1.97X	100% Match
1000	0.0113	0.0026	4.29X	100% Match
2000	0.0456	0.0051	8.88X	100% Match
4000	0.2187	0.0147	14.89X	100% Match
8000	0.9191	0.0274	33.51X	100% Match

Pipeline de Execução no Google Colab (GPU NVIDIA)

Para rodar os kernels CUDA na GPU de forma real – contornando a falta de GPU NVIDIA local – estruturamos um pipeline automático completo:



Configuração e Arquivos

- **Notebook Colab** configurado com acelerador **T4 GPU** ativo
- Upload de `baseline_cpu.cpp`, `metrics_cuda.cu` e `benchmark.py`

Automação via `benchmark.py`

- Compila a CPU com `g++` e a GPU com `nvcc`
- Executa testes de escalabilidade e valida precisão matemática (corretude 100%)
- Mede tempos reais e exporta o gráfico consolidado de speed-up (`curva_performance_cuda.png`)

Próximos Passos – Etapa Final

Para a consolidação da etapa de implementação final do projeto, planejamos as seguintes ações:



Otimizações no Kernel de Distância

Implementar *Tiling* com memória compartilhada para minimizar a banda de memória global e aumentar a reutilização de dados em cache.



Testes de Larga Escala

Avaliar o limite de memória da GPU e barramento PCIe escalando o dataset para **N=50.000** e **N=100.000** registros.



Uso de Streams no CUDA

Executar transferências de dados de forma assíncrona concorrentemente com o processamento dos Kernels – overlap de H2D/D2H para máxima utilização do hardware.



Relatório Final

Construção da monografia final descrevendo a engenharia dos kernels e análise comparativa detalhada de speed-up entre CPU e GPU.

Obrigado!

Abertos a dúvidas e sugestões.

Equipe

- Henrique M. M. Miranda
- Cindy Stephanie Gomes Rabelo
- Eduardo Dias Peixoto
- Luiany Goncalves Carvalho

Resumo das Conquistas

- Speed-up máximo de **33.51X** com 8.000 pontos
- Corretude de **100%** CPU vs GPU em todos os testes
- Validação contra scikit-learn com erro de **$\sim 2.28 \times 10^{-9}$**
- Pipeline automatizado no Google Colab com NVIDIA T4

Disciplina: Computação de Alto Desempenho – UFG
Instituto de Informática